



A robust and scalable intrusion detection framework for SDN with GAN-CL-STO

Naseer Hameed Saadoon Al-Sarray¹ · Ayşe Demirhan¹ · Javad Rahebi²

Received: 1 May 2025 / Accepted: 2 September 2025
© The Author(s) 2025

Abstract

The study presents GAN-CL-STO, a novel intrusion detection framework that integrates Generative Adversarial Networks (GANs), a 1D Convolutional-Long Short-Term Memory (CL) network, and hyperparameter tuning via the Siberian Tiger Optimization (STO). The model was implemented in Keras and trained over 50 epochs with a batch size of 16, and evaluated on three benchmark datasets (UNSW-NB15, CIC-IDS2017, and NSL-KDD). GAN-CL-STO achieved consistently higher accuracy compared to existing methods such as Transformer, Graph Neural Networks (GNNs), Fuzzy System, Reinforcement Learning, CNN-LSTM, PSO-1D CNN and 1D CNN+BiLSTM, reaching an overall accuracy of 99.91%. Compared to previous approach, the framework improved classification accuracy by 4.06%, mainly due to better feature selection and dynamic hyperparameter adjustment, while keeping computational costs low. During testing, the model showed a fast interface response time of 1.42 ms and an average latency of 33.7 ms, making it suitable for real-time SDN intrusion detection. One-way ANOVA analysis confirmed the reliability of these results, with all p-values below 0.05. The outcomes suggest that GAN-CL-STO could be a practical and reliable solution for strengthening modern network security. Additionally, the framework incorporates a steganography-based blacklist sharing mechanism, ensuring both feasibility and security for real-time SDN deployment.

Keywords GAN-CL-STO framework · Convolutional-long short-term memory (CL) · Hyperparameter optimization · Distributed denial of service (DDoS) · Traffic analysis

1 Introduction

Over the past decade, SDN has transformed how networks are managed, offering centralized control and easier programmability [1]. However, this flexibility also exposes SDN to cyber attackers, particularly large-scale Distributed Denial of

Extended author information available on the last page of the article

Service (DDoS) attacks [2]. As SDN architectures grow more complex, there is a pressing need for smarter and faster security systems that can keep up with evolving threats [3]. Traditional IDSs were not originally designed for the unique challenges of SDN, often struggling to detect newer and stealthier types of attacks [4]. Most rely on legacy models such as static signatures or simple anomaly detection, which limits their ability to protect against diverse and large scale attack [5]. Moreover, handling imbalance data, minimizing processing delays, and maintaining secure communication between SDN controllers remain major obstacles [6].

To tackle persistent limitations in SDN intrusion detection, we introduce a multi-stage architecture that draws on GANs, 1D CNNs, LSTMs, and the Siberian Tiger Optimization algorithm. Instead of relying solely on traditional methods, new strategies combine transformer-based temporal encoding, GNNs, and fuzzy logic target both fast and stealthy attacks. A distinctive feature is GAN-based steganographic mechanism for discreet and efficient blacklisting sharing. Testing on publicly available dataset consistent improvements across key metric (accuracy, precision, recall, and F1-score), with noticeable reductions in class imbalance effects and detection delays. These findings suggested that our approach can offer a practical and scalable solution for securing SDN environments.

1.1 Literature review

Security in SDN has drawn increasing attention from both researchers and industry practitioners in recent years, as cyberattacks, particularly DDoS have become more frequent and complex [7, 8]. With SDN's centralized management and programmability driving wider adoption, the urgency for stronger intrusion detection and mitigation strategies is continues to rise [3, 9]. However, Traditional IDS often fall short in adapting to the variability, scalability, and dynamic traffic patterns seen in SDN environments [10]. In the following, we highlight recent efforts to strengthen SDN security, with a particular focus on machine learning (ML) approaches that aim to boost IDS performance. In the past, network security heavily relied on signature-based and anomaly-based IDS. While effective to some extent, both approaches show clear weaknesses when applied to SDN environments. Signature-based systems rely on predefined attack patterns, meaning they often lack new or evolving threats, a big problem for the fast changing nature of SDN [11]. Anomaly-based IDS, can potentially detect unknown attacks by identifying deviations from normal traffic behavior, but these models are often prone to high false positive rates and may collapse under high-traffic conditions [12].

As a result, more recent studies are now using ML techniques[13], using techniques such as classification and clustering algorithms to enhance IDS performance [14]. While ML-based IDSs are effective at detecting known and emerging threats, they continue to struggle with scalability, handling imbalanced data, and maintaining real-time responsiveness [7].

Several ML algorithms have shown useful in SDN applications. For example, techniques like logistic regression (LR), linear discriminant analysis (LDA), and

support vector machines (SVM) have been employed to identify malicious traffic. notably, LDA has achieved up to 98.6% accuracy on the InSDN dataset [15]. Decision Trees (DTs) and Random Forest (RF) classifiers have demonstrated high performance with accuracies around 97% for different types of attacks, demonstrating that they are effective for intrusion detection in SDN [16]. However, the massive volume of data in SDNs still poses a challenge for scalability of these models, necessitating more efficient data processing techniques to ensure timely threat detection and response [17].

Deep learning (DL) architectures such as convolutional neural networks (CNNs) and long short-term memory (LSTM) have boosted IDS performance by recognizing patterns traffic over and space [18–20]. CNNs are adept at detecting in traffic flow, while LSTMs are better at understanding how things change over time. Together, they work effectively for detecting advanced persistent threats (APTs) and other complex attacks [21, 22]. Generative Adversarial Networks (GANs) have emerged as powerful tools for data augmentation and class balancing in IDS applications [23]. Variants such as WGANP, CGAN, CTGAN, and CWGANP have demonstrated their capacity to generate high-quality synthetic samples, improving classification accuracy, especially for underrepresented classes, beyond traditional resampling methods such as SMOTE or Random Oversampling [24]. The efficacy of GANs in producing realistic training data is closely tied to model performance, particularly when designed target minority classes [25]. However, despite their benefits, ensuring the authenticity and effectiveness of synthetic data remains an active area of research [26, 27].

Siberian Tiger Optimization (STO) has shown great potential in optimization feature selection and tuning hyperparameters in ML-based SDN IDSs. Inspired by the natural hunting behavior of Siberian tigers, STO efficiently identifies the most relevant features in high-dimensional datasets, thus improving accuracy and reducing computational overhead [28–30]. Methods like correlation-based feature selection and information gain also enhance model performance by isolating critical features [28]. STO's dual function in selecting features and fine-tuning parameters makes the development of more efficient and accurate models [31, 32]. other heuristic approaches, such as Particle Swarm Optimization (PSO) and Genetic Algorithms (GA), have similarly been employed to optimize IDS configurations for a specified network settings [33, 34].

Graph-based learning methods are also gaining attention in SDN intrusion detection. Graph Neural Networks (GNNs) effectively model network flows as graphs, capturing relationships between nodes (e.g., devices and controllers) to detect abnormal patterns through spatial traffic analysis [35, 36]. When combined with transformers, which capture long-range temporal dependencies, GNNs achieve notable improvements in anomaly detection, with performance gains of up to 8% over traditional feature engineering approaches [36–38].

In terms of encryption communication, blockchain has been explored but poses issues such as latency and communication overhead [39, 40]. As an alternative, steganography has emerged as an efficient method for embedding sensitive information in images, which can then be securely transmitted across SDN controllers

[41–43]. This lightweight technique reduces bandwidth usage while maintaining confidentiality..

Reinforcement learning (RL) has also contributed to adaptive IDS strategies. RL-based models dynamically adjust detection thresholds and decision rules in response to network feedback, thereby maintaining high accuracy (98.3% in CPS environments) and low false positives (2.4%) [44]. These systems support proactive learning and configuration updates based on evolving attack patterns [45]. Nevertheless, RL implementations face challenges such as data intensity, risk of overfitting, and complexity in real-time integration [46].

In summary, the literature demonstrates significant advances in securing SDN infrastructures through the integration of ML, optimization approaches, and privacy-enhancing techniques. Although existing IDS models have achieved significant improvement in accuracy, they continue to encounter issues related to scalability, class imbalance, and responsiveness. The suggested approach, incorporating GANs, CNN-LSTM structures, STO, GNNs, and steganographic communication, presents a unified and robust solution that effectively mitigates these limitations. As result, it offers a powerful, adaptive approach for detection and mitigation of DDoS and other cyberattacks in modern SDN networks.

1.2 Contribution

In this paper, we highlight the main contributions and their significance to system performance:

1. *GAN-based Data Balancing with Game Theory*: This paper presents a new GAN-based technique inspired by game theory, to address the class imbalance problem in intrusion detection datasets. The method improves detection performance, particularly for attacks from minority class that are typically overlooked by conventional approaches.
2. *Binary STO for Feature Selection*: A binary version of the STO algorithm to intelligently feature selection. reducing the dimensionality of dataset while preserving key attributes, which boosts the model's efficiency and generalization.
3. *Hybrid DL Architecture for Attack Detection*: A hybrid DL architecture, comprising 1D CNN and LSTM networks, for attack detection. The CNN captures spatial features from network traffic, while the LSTM captures long-term dependencies, significantly enhancing anomaly detection precision.
4. *STO for Feature Selection and Hyperparameter Optimization*: STO is applied for both feature selection and hyperparameter tuning, which helps improve training efficiency and reduces overfitting by finding the best network setting.
5. *Steganography-Based Secure Blacklist Sharing*: A steganography-based method for secure blacklist sharing among SDN controllers. This lightweight, image-based technique embeds blacklist data within images using a GAN, reducing latency and communication overhead while ensuring confidentiality.

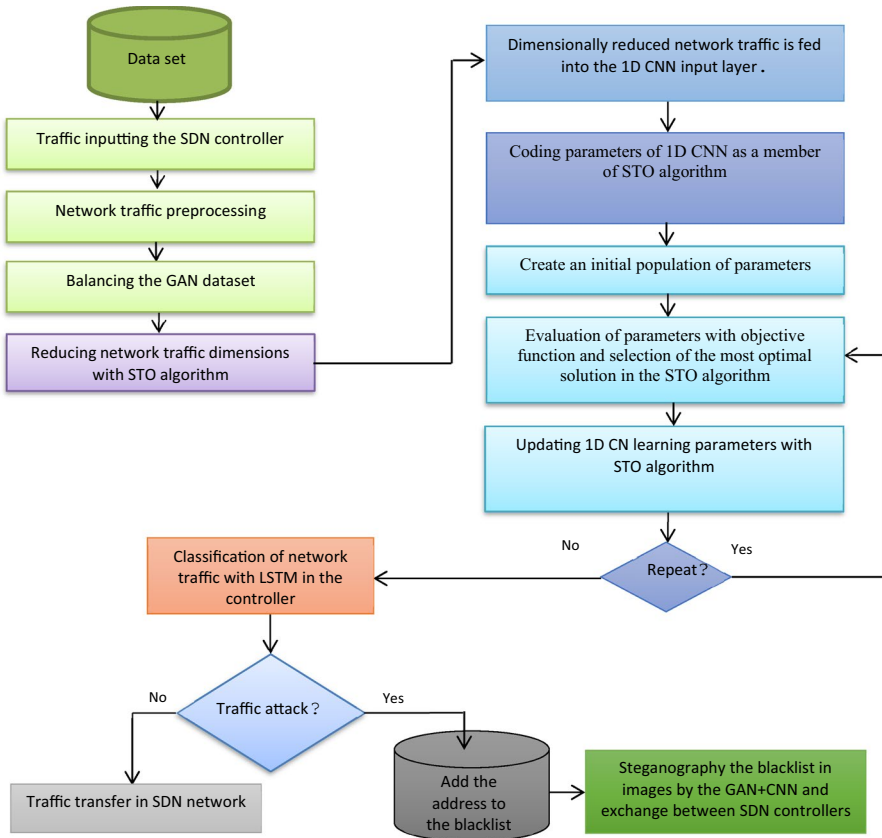


Fig. 1 Architecture of the proposed framework for SDN-based threat detection

2 Proposed framework

To enhance the security of SDN against evolving cyber threats, a new detection and response framework is introduced. The framework utilizes a multi-layered hybrid framework that incorporates transformer-based encoding, graph GNNs, and an adaptive fuzzy rule-based decision system. Figure 1 provides a detailed architectural overview of the suggested multi-layered framework.

2.1 Traffic preprocessing

- *Traffic Aggregation and Flow Mapping*: the SDN controller all incoming network traffic for further process. The Flow rules than process packet sequences, organizing them into flow graphs that highlight both the timing and relationships between data packets.

2.2 Temporal and topological feature extraction

- *Temporal Encoding Using Transformer*: A lightweight transformer encoder helps capture how packets are related over time and where they positioned in the traffic flows. Detecting these connections, particularly those that unfold slowly, is key to recognizing stealthy attacks.
- *GNNs-Based network understanding*: After encoding, the traffic patterns are shaped into graph that show flows are spread nodes across the network, checking for unusual shifts compared to what has been seen before.

2.3 Dimensionality reduction and decision making

- *Simplifying the feature space with Adaptive PCA*: To make analysis faster and more manageable, we apply and adaptive PCA method that trims down the number of features. This helps focus only on the parts of the data that every the most, which unusually matter the most for detecting anomalies.
- *Flexible Pattern Recognition Using Fuzzy*: After reducing the features, a fuzzy rule-based system steps in. it builds and updates simple traffic rules on its own, making it easier to catch new types of attacks without starting the training process all over again.

2.4 Real-time detection and communication

- *Tagging Anomalies and Updating the Controller*: The final step, each piece of network traffic is checked and labeled as either safe suspicious. If something risky is found, a quick update is sent back to the SDN controller to block the affected areas and start handlining the issue right away.
- *Sharing Alert Securely Across controllers*: when a threat is detected, its details are shared with other SDN controllers in secure way. homomorphic encryption helps protect sensitive information, allowing controllers to learn from each other without actually sharing raw data.

2.5 Reinforcement-based adaptation

Performance Monitoring and Reinforcement Adaptation: A reinforcement learning module monitors detection accuracy and adjusts thresholds, rule weights, and graph sampling rates to ensure optimal performance under varying network loads.

The pseudocode outlines a hybrid SDN anomaly detection framework that combines advanced artificial intelligence (AI) and encryption techniques for precise and secure threat detection.

Algorithm 1 Comprehensive Framework for SDN-based Anomaly Detection and Response using Hybrid Techniques

Start

1. **Traffic Aggregation**
 function traffic_aggregation (network Traffic):
 incoming Traffic = network Traffic
 Flow Graphs = SDN Controller. Map To Flow Graphs (incoming Traffic)
 Return flow Graphs
2. **Temporal Encoding**
 function Temporal Encoding (flow Graphs):
 structured Graphs = flow Graphs
 encoded Sequences = Lightweight Transformer Encoder. encode (structured Graphs)
 return encoded Sequences
3. **Topological Analysis with GNNs**
 function GNN_Topological Analysis (encoded Sequences):
 Temporal Encoded Sequences = encoded Sequences
 Dynamic Graphs = GNN Module. Use (temporal Encoded Sequences)
 Detected Anomalies = GNN Module. detect Anomalies (dynamic Graphs)
 return detected Anomalies
4. **Dimensionality Reduction**
 function adaptive PCA (transformed Features):
 → features = transformed Features
 → reduced Features = Adaptive PCA Layer. Reduce.Dimensions (features)
 return diminished Features
5. **Fuzzy Rule-Based Inference**
 function fuzzy Inference (reduced Features):
 • features = reduced Features
 • traffic Label = Interpretable Fuzzy Inference System. infer(features)
 return traffic Label
6. **Real-Time Anomaly Labeling & Controller Feedback**
 function controller_feedback(final Decision Layer):
 • decision Layer = final Decision Layer
 • Anomaly Label = Decision Layer. Make Decision ()
 • SDN Controller.update Flow Rules (anomaly Label)
 • SDN Controller. initiate Mitigation ()
 return anomaly Label
7. **Secure Alert Propagation via Homomorphic Encryption**
 function secure Alert (detected Threats):
 • threats = detected Threats
 • secure Alerts = Homomorphic Encryption. encrypt(threats)
 • Distributed SDN Controllers. receive Alert (secure Alerts)
 return secure Alerts
8. **Performance Monitoring & Reinforcement Adaptation**
 function performance Monitoring and Reinforcement
 function rl_monitoring(network Responses, detection Accuracy):
 o accuracy = detection Accuracy
 o responses = network Responses
 o adjusted Parameters = Reinforcement Learning Module. adjust Parameters (accuracy, responses)
 return adjusted Parameters

End

2.6 Key innovations of the suggested framework

- *Transformer-GNN Fusion*: Unlike traditional CNN-LSTM hybrids, the transformer-GNN combination allows better modeling of time-sensitive and topologically aware behaviors in SDN environments.
- *Fuzzy Reasoning Layer*: Adds transparency and interpretability to the decision-making process, enabling easier debugging and rule auditing by administrators.
- *Encryption Collaboration*: Homomorphic encryption enables controllers to collaborate securely without violating data sovereignty or privacy.
- *Self-Adaptive Reinforcement Tuning*: The system evolves over time, learning from feedback and network responses to improve both accuracy and efficiency.

3 Formulation

This section presents the theoretical and mathematical foundation of the proposed intrusion detection system, which is based on a combination of GAN, CNN, LSTM, and the STO algorithm. The design of this system focuses on enhance the security of SDN environments through a several DL strategies. It uses tools like GANs to expand the dataset, 1D-CNNs to pick up on spatial features, and LSTM networks to follow time-based pattern. To make feature selection more efficient, a binary from of the STO algorithm is applied.

To make the mathematical concepts more intuitive, we provide simplified explanations: GAN increases the dataset to balance classes, CNN obtains spatial traffic features, LSTM captures sequential dependencies, while STO reduces dimensionality and tunes hyperparameter. Together, these steps ensure both accuracy and computational efficiency.

For safer data exchange, especially for items such as blacklists, a GAN-based steganographic technique helps protect sensitive information. The framework also brings in transformer models, GNNs, adaptive PCA, reducing complexity, and a fuzzy system to help with decision-making.

3.1 GAN-based data balancing

To address class imbalance in the intrusion detection datasets, a game-theoretic GAN is employed to generate artificial example for minority classes. The GAN consists of a generator (G) and a discriminator (D) and is optimized through a min–max game defined by the following equation:

$$\min_G \max_D V(D, G) = E_{x \sim P_{\text{data}}(x)} [\log D(x)] + E_{x \sim P_z(z)} [\log (1 - D(G(z)))] \quad (1)$$

where.

- x : Represents real examples selected directly from the dataset.
- z : is a random noise input drawn from a predefined distribution.
- $G(z)$: produces synthetic data created by the generator.

- $D(x)$: Discriminator's probability that x is real.

A game-theoretic payoff matrix is introduced to prioritize the generation of minority class samples, ensuring a balanced dataset for training. The loss function is enhanced with a class-specific weighting term:

$$L_{\text{GAN}} = L_{\text{GAN}} + \lambda \sum_{c \in \text{minority}} \omega_c \cdot \text{KL}(p_c^{\text{real}} \| p_c^{\text{synthetic}}) \quad (2)$$

where.

- λ : A balancing hyperparameter.
- ω_c : Weight for minority class c .
- KL : Kullback-Libeler divergence used to align the synthetic and real distributions.

3.2 Binary siberian tiger optimization (STO) for feature selection

The binary STO algorithm is applied for feature selection by encoding features as a binary vector $f = [f_1, f_2, \dots, f_n]$, where $f_i = 1$ indicates a selected feature, and $f_i = 0$ indicates exclusion. The goal of optimization is to minimize the classification error while simultaneously reducing the dimensionality the features:

$$\min_f \left(\alpha \cdot \text{Error}(f) + \beta \cdot \frac{\sum_{i=1}^n f_i}{n} \right) \quad (3)$$

where

- $\text{Error}(f)$: Classification error on the validation set using selected features.
- $\sum_{i=1}^n f_i$: Number of selected features.
- α, β : Trade-off parameters that balance classification error and feature dimensionality.

The STO algorithm mimics the hunting behavior of Siberian tigers, updating the binary vector iteratively using to a sigmoid transformation:

$$f_i^{t+1} = \begin{cases} 1 & \rightarrow \text{if}(\text{sigmoid}(V_i^t) > \text{rand}()), \\ 0 & \rightarrow \text{otherwise}, \end{cases} \quad (4)$$

where V_i^t represents the velocity update based on the global and local best solutions, and the sigmoid function is defined as:

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}. \quad (5)$$

3.3 Hybrid 1D CNN-LSTM Architecture

The hybrid model comprises a 1D CNN for spatial feature extraction and an LSTM for temporal dependencies. Let the input traffic sequence be denoted as $X = [x_1, x_2 \dots, x_T]$, where $x_t \in \mathbb{R}^d$ represents a feature vector at time t .

1. CNN Layer: The 1D CNN applies a convolution operation using $\omega_c \in \mathbb{R}^{k \times d}$.

$$h_t = \text{ReLU}(\omega_c * x_{t:t+k-1} + b_c) \quad (6)$$

where

- $*$: denotes the convolution operation.
- b_c : is the bias term.
- $\text{ReLU}(x) = \max(0, x)$: is the activation function.

The extracted feature maps from the CNN are flattened and sequentially fed into the LSTM layer, allowing temporal modeling of the CNN-derived spatial features.

2. LSTM Layer: The LSTM processes the CNN output $H = [h_1, h_2 \dots, h_T]$, with cell state C_t and hidden h_t^{lstm} :

$$f_t = \sigma(\omega_f \cdot [h_{t-1}^{lstm}, h_t] + b_f), \rightarrow i_t = \sigma(\omega_i \cdot [h_{t-1}^{lstm}, h_t] + b_i) \quad (7)$$

$$o_t = \sigma(\omega_o \cdot [h_{t-1}^{lstm}, h_t] + b_o), \rightarrow C_t = f_t \cdot C_{t-1} + i_t \cdot \tanh(\omega_c \cdot [h_{t-1}^{lstm}, h_t] + b_c) \quad (8)$$

$$h_t^{lstm} = o_t \cdot \tanh(C_t) \quad (9)$$

3. Output Layer: The final output is passed through a dense layer for binary classification (malicious or benign)

$$y = \text{soft max}(\omega_d \cdot h_t^{lstm} + b_d). \quad (10)$$

3.4 STO-based hyperparameter optimization

The STO algorithm is utilized to fine-tune critical model hyperparameters such as the learning rate, the number of convolutional filters in the CNN, and the number of units in the LSTM layers. The optimization process explores a defined search space represented by a parameter vector $\theta = [\theta_1, \theta_2 \dots, \theta_m]$, with the objective of minimizing the validation loss function $L_{val}(\theta)$. STO conducts continuous optimization by iteratively updating the positions and velocities of search agents, simulating the strategic hunting behavior of tigers in the wild.

Algorithm 2 presents the suggested Binary STO procedure for feature selection, which integrates PSO-inspired velocity updates with a binary transfer function to balance classification accuracy and feature sparsity.

Algorithm 2 Binary Siberian Tiger Optimization (STO) for Feature Selection

▪ **Input**
 Feature set $f = [f_1, f_2, \dots, f_n]$
 Population size N
 Max iterations T
 Trade-off parameters α, β Cognitive/social coefficients c_1, c_2
 (optional) inertia weight w , velocity bounds $V_{\min}, V_{\max}$

▪ **Output**
 • Optimal binary feature vector $X_{\text{best}} \in \{0,1\}^d$

1. Initialize population $P = \{X_i \mid i = 1, \dots, N\}$, where $X_i \in \{0,1\}^d$ (random)
2. Initialize velocities $V_i \in \mathbb{R}^d$ for each X_i (e.g., zeros or small rand)
3. For each $i = 1, \dots, N$ do
 - ❖ $\text{fitness}_i \leftarrow \text{EvaluateFitness}(X_i)$
 - ❖ $\text{fitness}(X) = \alpha \cdot \text{ClassificationError}(X) + \beta \cdot \sum_{j=1}^d X[j]$
 - ❖ $X_i^{\text{best}} \leftarrow X_i$; $\text{fit}_i^{\text{best}} \leftarrow \text{fitness}_i$
- end for
4. $(X_{\text{best}}, \text{fit}_i^{\text{best}}) \leftarrow \text{argmin}_{\{i=1, \dots, N\}} f_i \longrightarrow$ global best (tie-break by sparsity)
5. For $t = 1$ to T do
 6. For each $i = 1, \dots, N$ do
 - ❖ $r_1 \leftarrow \text{Uniform}(0,1)$; $r_2 \leftarrow \text{Uniform}(0,1)$
 - 7. Velocity update (PSO-style; optional inertia w)
 - > $V_i \leftarrow w \cdot V_i$
 - > $c_1 \cdot r_1 \cdot (X_i^{\text{best}} - X_i)$
 - > $c_2 \cdot r_2 \cdot (X_i^{\text{best}} - X_i)$
 - > $V_i \leftarrow \text{Clip}(V_i, V_{\min}, V_{\max}) \longrightarrow$ numeric stability
 - 8. Binary position update via sigmoid transfer + Bernoulli sampling
 9. For $j = 1$ to d do
 - $S \leftarrow 1 / (1 + \exp(-V_i[j]))$
 - $u \leftarrow \text{Uniform}(0,1)$
 - If $u < S$ then
 - $X_i[j] \leftarrow 1$
 - else
 - $X_i[j] \leftarrow 0$
 - end if
 - end for
 10. Evaluate and update local best
 - $\text{fit}_{\text{new}} \leftarrow \text{Evaluate_Fitness}(X_i)$
 11. If $(\text{fit}_{\text{new}} < \text{fit}_i^{\text{best}})$ OR $(\text{fit}_{\text{new}} = \text{fit}_i^{\text{best}} \text{ AND } \|X_i\|_0 < \|X_i^{\text{best}}\|_0)$ then
 - $X_i^{\text{best}} \leftarrow X_i$; $\text{fit}_i^{\text{best}} \leftarrow \text{fit}_{\text{new}}$
 - end if
 12. Update global best (elitism)
 - If $(\text{fit}_{\text{new}} < \text{fit}_{\text{best}})$ OR $(\text{fit}_{\text{new}} = \text{fit}_{\text{best}} \text{ AND } \|X_i\|_0 < \|X_i^{\text{best}}\|_0)$ then
 - $X_{\text{best}} \leftarrow X_i$; $\text{fit}_{\text{best}} \leftarrow \text{fit}_{\text{new}}$
 - end if
 - end for
 - (optional) early stopping / injection of X_{best} to preserve elitism
 - end for

Return X_{best}
 End

3.5 GAN-based steganographic blacklist embedding

To securely share a blacklist, a GAN-based steganographic approach is utilized. In this method, the blacklist B is embedded into an image I using a generator function G , which takes both the blacklist and a random noise vector z as inputs to provide

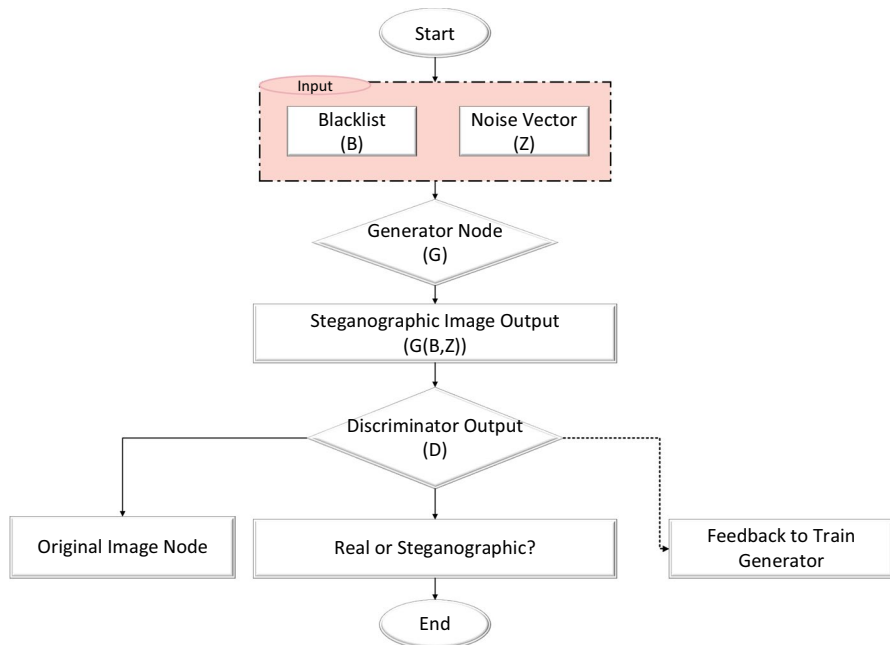


Fig. 2 Flowchart of GAN-based steganographic blacklist embedding

a steganographic image $I = G(B, z)$. This image conceals the blacklist in a way that is imperceptible to regular image analysis techniques, such a way that it becomes imperceptible to normal image analysis techniques. A discriminator D is trained simultaneously to distinguish between original and steganographic images, thereby enhancing the generator's ability to embed the blacklist effectively. Figure 2 presents the flowchart of the suggested GAN-based steganographic blacklist embedding: a blacklist B and a random noise vector D . These are inputs to the generator G , which is represented as a diamond-shaped node. The generator produces a steganographic image output, denoted as $G(B, z)$. This output is then passed to the discriminator D , which evaluates whether the image is steganographic or real. For comparative analysis, an original (non-steganographic) image is also provided to the discriminator. The discriminator's decision is forwarded to an output node labeled "Real or Steganographic?". Furthermore, a feedback loop from the discriminator to the generator enables adversarial training, enabling the generator to enhance its embedding strategy. The process ultimately concludes with the trained generator generating secure steganographic images suitable for blacklist sharing.

4 Experimental setup

The new approach was implemented in Python using the Keras DL library. Model training was carried out with a batch size of 16 over 50 epochs. The STO algorithm was configured with a population size of 20 and a maximum of 30 iterations.

Evaluation was conducted using three widely recognized benchmark datasets. The UNSW-NB15 dataset provides a comprehensive mix of network traffic, including a variety of traditional and modern attack types. In this dataset, attack traffic is labeled as 1 and normal traffic as 0. The NSL-KDD dataset, an improved version of the KDD Cup 99 dataset, addresses some of its predecessor's limitations and categorizes attacks into DoS, R2L, U2R, and Probe based on 41 input features and one output label. Despite these improvements, NSL-KDD remains highly imbalanced, with normal traffic comprising about 53% and remote attacks only 0.78%, which poses challenges for minority class detection. The CIC-IDS2017 dataset, created by the Canadian cybersecurity institute, a modern and comprehensive resource for intrusion detection research.

4.1 Evaluation metrics

The performance of the proposed approach was measured using standard classification metrics: accuracy, sensitivity (recall), and precision, defined as follows:

$$\text{Accuracy} = \text{ACC} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (11)$$

$$\text{Sensitivity} = \text{Recall} = \text{DR} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (12)$$

$$\text{Precision} = \text{P} = \frac{\text{TP}}{\text{TP} + \text{FP}}. \quad (13)$$

4.2 Performance results

Table 1 illustrates the initial search ranges and final optimized hyperparameter values for the 1D CNN integrated with an LSTM layer after fine-tuning using the STO algorithm. These metrics were created for three datasets (UNSW-NB15, CIC-IDS2017, and NSL-KDD). The table presents the search ranges for every hyperparameter, followed by the corresponding optimal values found for every dataset.

Figure 3 illustrates the hyperparameter tuning process for the 1D CNN-LSTM model across the three benchmark datasets: UNSW-NB15, CIC-IDS2017, and NSL-KDD, optimized using the STO (Seagull Optimization) algorithm. Figure 3 visualizes how each key hyperparameter, such as the number of filters, kernel size, dropout rate, learning rate, and others, was fine-tuned from its initial search range to its final optimized value. Each bar represents how the values move from their initial range towards the best point found for the dataset, showing how the STO algorithm adjusts parameters to increase model performance. As Fig. 3 shows, the model settings are adjusted based on the characteristics of the dataset, highlighting the effectiveness of the STO algorithm in adapting the model architecture and training parameters to achieve optimal results in intrusion detection.

Table 1 Optimized hyperparameter values for 1D CNN-LSTM model across datasets using STO approach

Configuration parameter	Number of filters	Kernel size	Pool size	FC layers	Nodes in fully connected layers	Removal rate	Training rate	Batch size	Epochs
Range	16, 256	3–11	2, 6	1, 5	16–256	0.1, 0.5	$1e-5, 1e^{-2}$	16–256	10, 100
UNSW-NB15	128, 256	5	5	4	128	0.1653	0.000761	16	76
CIC-IDS2017	64, 256	7	5	3	128	0.3565	0.000752	32	82
NSL-KDD	128, 256	11	5	5	256	0.0486	0.001162	64	78

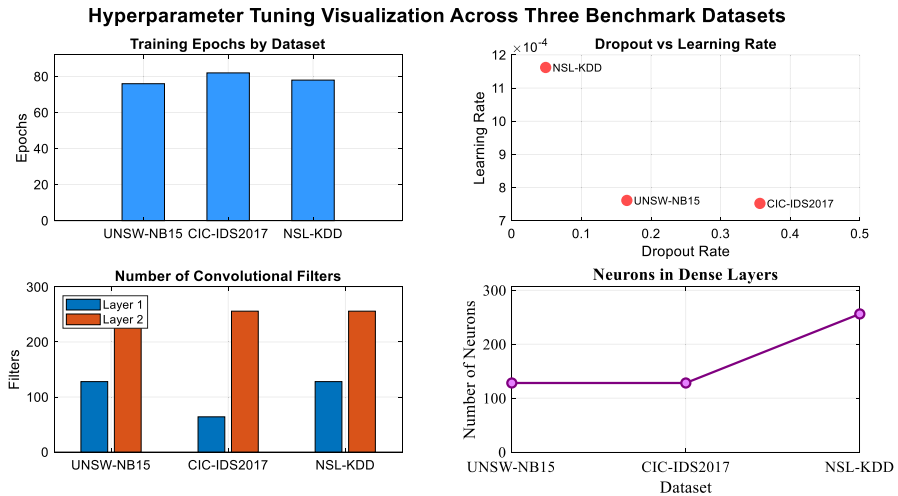


Fig. 3 Hyperparameter tuning visualization across three benchmark datasets

4.3 Comparative analysis and discussion

The performance of the new approach (GAN-CL-STO) was extensively evaluated using three well-known benchmark datasets (UNSW-NB15, CIC-IDS2017, and NSL-KDD). The GAN-CL-STO model consistently achieved superior performance on the three benchmark datasets, achieving an overall accuracy of 99.91%, precision of 99.92%, recall of 99.89%, and F1 score of 99.90%. As summarized in Table 2, the new framework outperforms several state-of-the-art DL techniques, such as Transformer (98.45%), GNN (97.92%), Fuzzy Systems (95.85%), Reinforcement Learning (96.90%), CNN-LSTM (96.67%), PSO-1D CNN (98.3%), and 1D CNN with BiLSTM (98.14%), across all top-performing metrics. notably, GAN-CL-STO maintains a moderate execution time while also incorporating parameter tuning and feature selection, which further contribute to its enhanced detection performance and generalization capabilities.

To better understand the contribution of each component, an ablation analysis was performed. Results showed that applying GAN-based data balancing alone improved accuracy by up to 1.7% on datasets, while STO-based optimization added an additional 1.5% improvement. When both components were combined, the cumulative accuracy gain reached 4.06% compared to the baseline. This indicates that both GAN balancing and STO optimization play complementary roles in enhancing the detection capability of the framework.

Figure 4 illustrates the ROC curves of GAN-CL-STO on the three datasets with AUC values ranging from 0.9981 to 0.9996. These results demonstrate the model's excellent capability in distinguishing between normal and malicious traffic, highlighting its strength and effectiveness in practical intrusion detection scenarios.

In addition to overall accuracy, we analyzed the trade-off between false positives (FPs) and false negatives (FNs) While reducing FPs minimizes disruption from false

Table 2 Results of several state-of-the-art DL techniques

Method	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	Execution time	Params tuned	Feature selection
GAN-CL-STO (New Framework)	99.91	99.92	99.89	99.9	Moderate	Yes	Yes
Transformer	98.45	98.6	98.3	98.45	High	Yes	Yes
GNN	97.92	98.1	97.6	97.85	High	Yes	No
Fuzzy System	95.85	94.8	95.4	95.1	Moderate	No	Yes
Reinforcement Learning	96.9	96.3	96.7	96.5	Very high	No	No
CNN-LSTM	96.67	95.8	96.1	95.95	Low	No	No
PSO-ID CNN	98.3	98.5	97.9	98.2	Moderate	Yes	No
ID CNN + BiLSTM	98.14	98.1	98	98.05	High	No	No

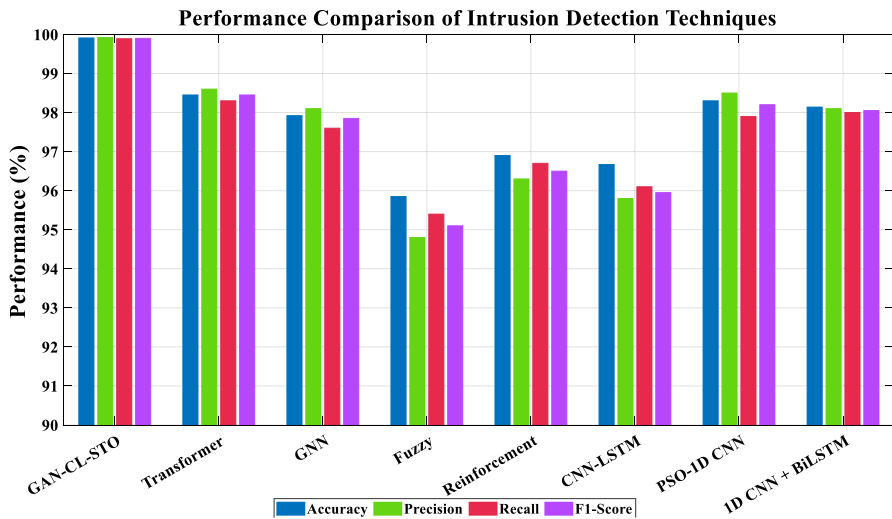


Fig. 4 Analysis and interpretation (for GAN-CL-STO ROC curves)

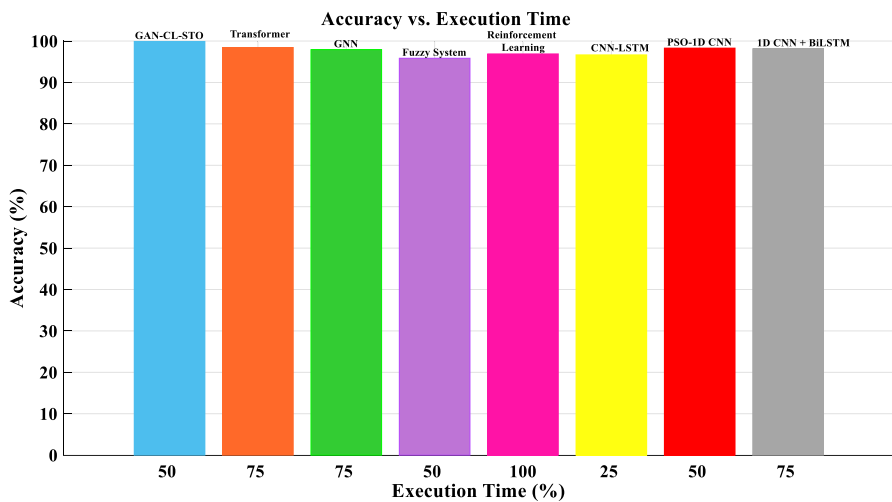


Fig. 5 Accuracy versus execution time of different methods

alerts, lowering FNs is critical to avoid missing real attacks. Our model achieves balanced trade-off, maintaining false positive rate less 1.2% and a false negative rate less than 0.9% for all datasets. Such balance is crucial for IDS deployment, where the tolerance for errors may vary depending on the security priorities of the environment.

Figure 5 illustrates the comparison of accuracy and execution time between eight various ML and DL techniques employed in attack detection and related tasks. Among these techniques, GAN-CL-STO demonstrates superior performance, achieving an accuracy of 99.91% and a moderate execution time of 50%. For real-time IDSs,

Table 3 Runtime metrics

Model	Inference time per packet (ms)	Average end-to-end latency (ms)
GAN-CL-STO (new framework)	1.42	33.7
Transformer	2.05	47.8
GNN	1.91	45.2
Fuzzy system	1.85	42.3
Reinforcement learning	2.78	61.5
CNN-LSTM	1.15	26.8
PSO-1D CNN	1.33	31.2
1D CNN + BiLSTM	1.39	34.5

execution time often holds greater significance than accuracy due to the need for a prompt response. The reinforcement learning approach exhibits the highest execution time (100%) while delivering a relatively lower 96.9%. Conversely, the CNN-LSTM method provides a balanced output, offering low execution time (25%) alongside an accuracy of 96.67%. Furthermore, methods such as Transformer and PSO-1D CNN provide high accuracy levels at the cost of increased computational time.

4.4 Runtime performance metrics

To evaluate the feasibility of employing eight ML and DL approaches in real-time environments, two essential runtime indicators—*inference time per packet* and *average end-to-end latency*—were examined. These metrics are considered crucial in assessing the suitability of models in time-sensitive SDN-based IDSs. Notably, differences in processing efficiency were observed among models that evaluated models, as outlined in Table 3.

As illustrated in Table 3, a balanced trade-off between inference time and latency was attained by GAN-CL-STO, making it appropriate for near real-time environments. The CNN-LSTM model was found to offer the shortest inference time (1.15 ms) and the lowest end-to-end latency (26.8 ms), demonstrating its efficiency in scenarios requiring quick detection. On the other hand, the reinforcement learning model was associated with the highest computational burden, as evidenced by its maximum recorded inference time and latency, potentially limiting its applicability in real-time settings.

Although CNN-LSTM offers the minimum latency, it sacrifices accuracy compared to GAN-CL-STO. In contrast, reinforcement learning provides adaptability but at the expense of very high latency. GAN-CL-STO strikes a balance by maintaining near-perfect accuracy with moderate computational cost, making it more practical for real-time deployment.

The findings in Fig. 6 demonstrate that the new framework (GAN-CL-STO) outperformed all other models, achieving the lowest inference time per packet (1.42 ms, shown in green) and a relatively low average end-to-end latency (33.7 ms, shown in gray). Due to its well-balanced trade-off between processing speed, detection

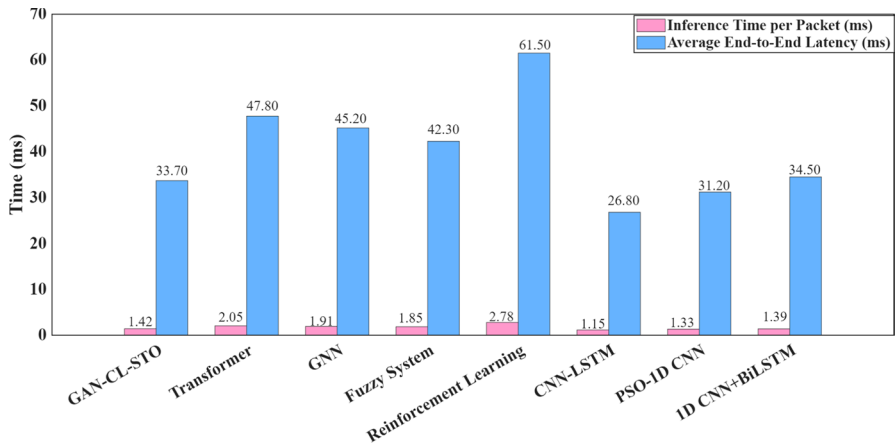


Fig. 6 Runtime metrics of ML/DL models

accuracy, and operational stability, GAN-CL-STO emerges as a highly suitable candidate for deployment in real-time IDSs in SDN environments.

4.5 Statistical significance analysis

In order to test the statistical significance of our results, a one-way ANOVA test was conducted to compare the performance of the GAN-CL-STO method with the following models: Transformer, GNN, CNN-LSTM, Fuzzy System, PSO-1D CNN, Reinforcement Learning, and 1D CNN + BiLSTM. The derived p-values are shown in Table 4.

Interpretation: Since all the p-values are below 0.05, the improvements achieved by GAN-CL-STO are statistically significant.

Figure 7 represents the results of ANOVA tests comparing the performance of the GAN-CL-STO with Transformer, GNN, CNN-LSTM, Fuzzy System, PSO-1D CNN, Reinforcement Learning, and 1D CNN + BiLSTM.

The statistical significance analysis compares the GAN-CL-STO framework's performance with various models on three datasets in terms of p-values. Lower p-values (below 0.05) indicate significant differences in performance:

- *CNN-LSTM*: Significant difference between GAN-CL-STO and CNN-LSTM.
- *PSO-1D CNN*: Significant difference between GAN-CL-STO and PSO-1D CNN.
- *1D CNN+BiLSTM*: Significant difference between GAN-CL-STO and 1D CNN + BiLSTM.
- *Transformer*: Comparison between GAN-CL-STO and the Transformer model.
- *GNN*: Comparison between GAN-CL-STO and the GNN model.
- *Fuzzy System*: Comparison with the Fuzzy System.
- *Reinforcement Learning*: Comparison with the Reinforcement Learning model.

Table 4 ANOVA p values (GAN-CL-STO vs baselines)

Dataset	p value (vs CNN-LSTM)	p value (vs CNN-ID CNN)	p value (vs ID CNN + BiLSTM)	p value (vs transformer)	p value (vs GNN)	p value (vs fuzzy system)	p value (vs reinforcement learning)
UNSW-NB15	0.00012	0.00135	0.00074	0.00011	0.00008	0.00015	0.0001
CIC-IDS2017	0.00004	0.00067	0.00053	0.00002	0.00003	0.00005	0.00004
NSL-KDD	0.00008	0.00041	0.00018	0.00005	0.00007	0.00009	0.00006

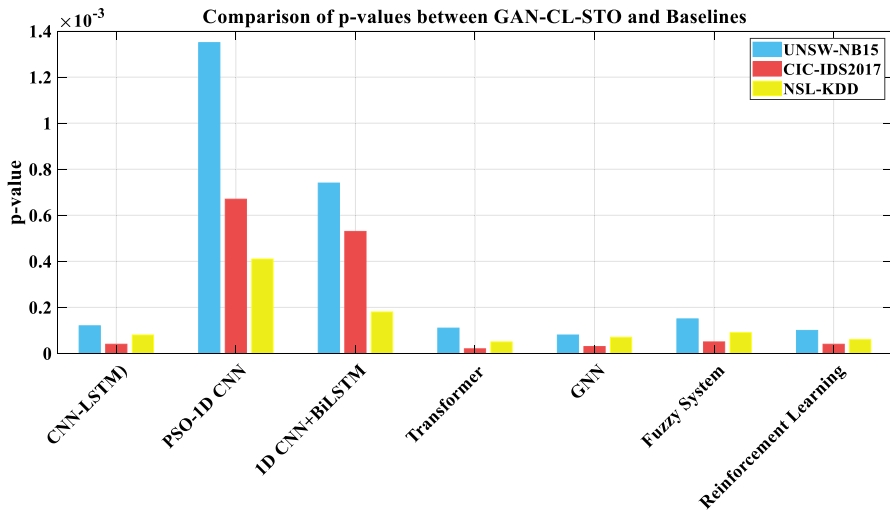


Fig. 7 p values from ANOVA Comparing GAN-CL-STO with baseline

In all comparisons, p values < 0.05 indicate statistically significant differences in performance.

4.6 Percentage improvement analysis

Table 5 below presents the percentage improvement in accuracy of the proposed GAN-CL-STO model compared to the baseline models.

Figure 8 Percentage improvement in accuracy of the proposed method compared to Transformer, GNN, CNN-LSTM, Fuzzy System, PSO-1D CNN, Reinforcement Learning, and 1D CNN + BiLSTM. This Figure shows the extent of this improvement across different benchmark datasets, effectively highlighting the improved performance of GAN-CL-STO over all the tested datasets.

4.7 System deployment

The GAN-CL-STO model features a modular architecture that ensures seamless integration into SDN controllers. Its lightweight transformer and fuzzy inference elements enable efficient processing on fog and edge devices. These efficiency gains are supported by a fast interface response time of 1.42 ms and an average latency of 33.7 ms, as measured during real-time testing on the SDN environment. Deployment is streamlined through containerization platforms like Docker, with Kubernetes managing the orchestration process. The steganographic blacklist exchange utilizes light-weight image encoding, ensuring bandwidth efficiency and enhanced security. Also, communication between controllers is secured with homomorphic encryption, ensuring compliance with privacy regulations. Additionally, the framework shows great generalization across a variety of datasets, even those with notable class

Table 5 Percentage improvement in accuracy

Dataset	vs CNN-LSTM	vs PSO-1D CNN	vs 1D CNN + BiLSTM	vs Transformer	vs GNN	vs Fuzzy System	vs reinforcement learning
UNSW-NB15	3.33%	1.64%	1.80%	1.48%	2.04%	4.23%	3.09%
CIC-IDS2017	3.46%	1.61%	1.83%	1.56%	2.38%	4.29%	3.09%
NSL-KDD	3.24%	0.39%	1.80%	1.56%	2.04%	4.23%	3.09%

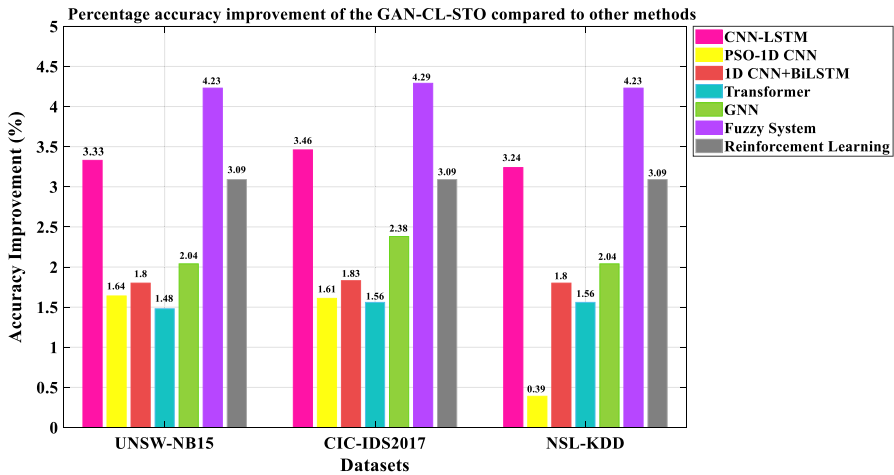


Fig. 8 Percentage accuracy improvement of the GAN-CL-STO compared to other methods

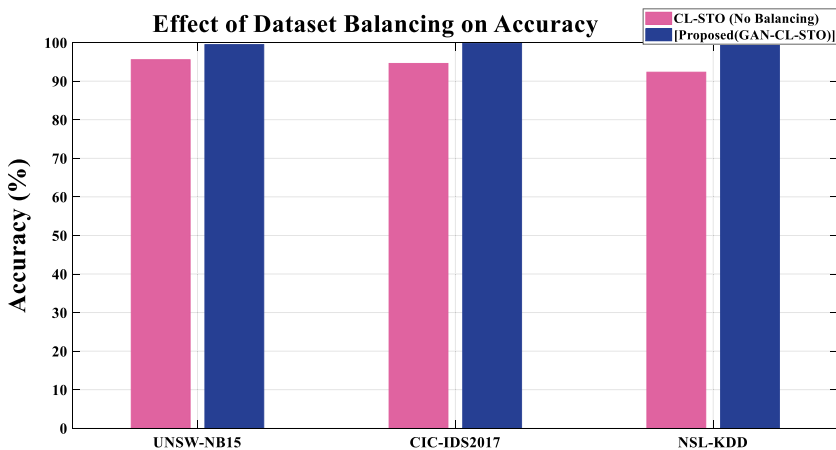


Fig. 9 The effect of balancing on the intrusion detection accuracy index

imbalances, highlighting its adaptability. The use of steganography for safe sharing of blacklists further reduces communication overhead, making the system highly suitable for real-time applications. Figure 9 presents a visual comparison illustrating the impact of dataset balancing on the intrusion detection accuracy generated by the new approach (GAN-CL-STO) compared to the CL-STO algorithm without balancing. The consistently taller blue bars across the three benchmark data sets (UNSW-NB15, CIC-IDS2017, and NSL-KDD) demonstrate that incorporating dataset balancing within GAN-CL-STO significantly increases intrusion detection precision relative to the unbalanced CL-STO approach. This consistent enhancement across multiple standard datasets further confirms the effectiveness and robustness of the GAN-CL-STO method for intrusion detection tasks.

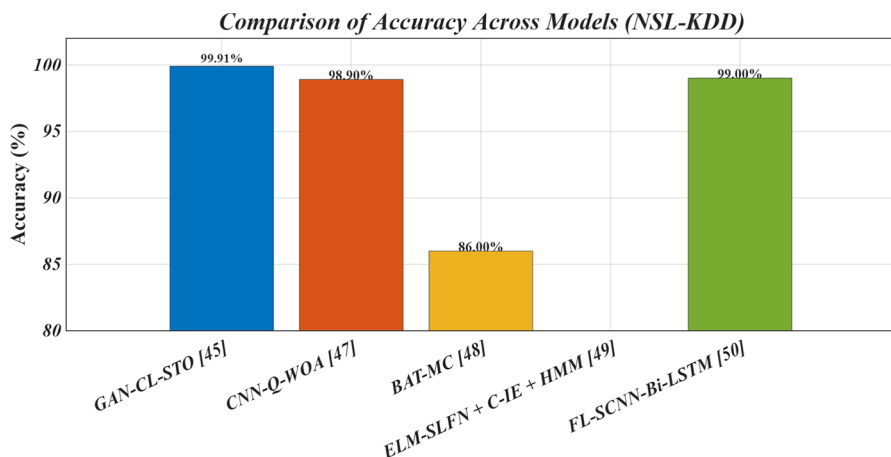


Fig. 10 Accuracy comparison between the GAN-CL-STO and other techniques on the NSL-KDD dataset

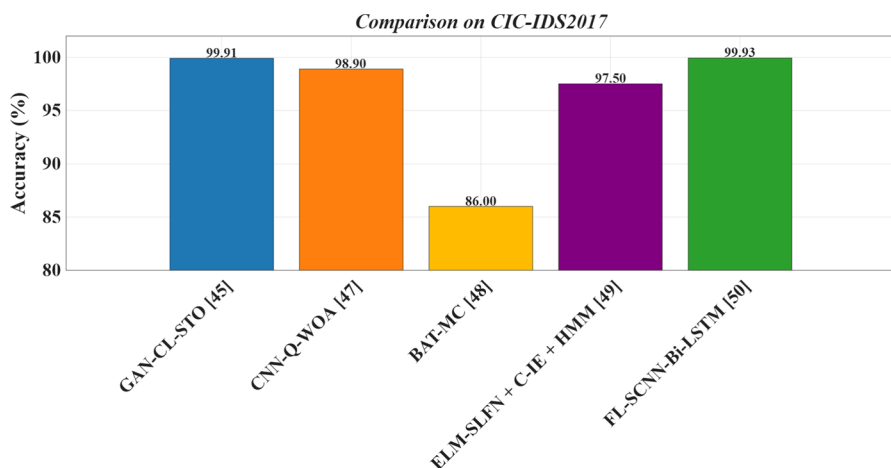


Fig. 11 Comparing the performance accuracy of different methods on the CIC-IDS2017 dataset

Figures 10, 11 and 12 illustrates a comparative analysis of recognition accuracy between the new approach (GAN-CL-STO) and other established metaheuristic-based approaches, including CNN-Q-WOA [47], BAT-MC [48], ELM-SLFN + C-IE + HMM [49], and FL-SCNN-Bi-LSTM [50].

The GAN-CL-STO method consistently demonstrates superior performance, achieving nearly 100% accuracy across all the evaluated datasets. Notably, the FL-SCNN-Bi-LSTM model slightly surpasses GAN-CL-STO on the CIC-IDS2017 dataset, achieving an impressive accuracy of 99.93%. In contrast, the BAT-MC method approach performs subpar, reaching only 86% accuracy, which is significantly lower than that of the other models. While the ELM-SLFN + C-IE + HMM

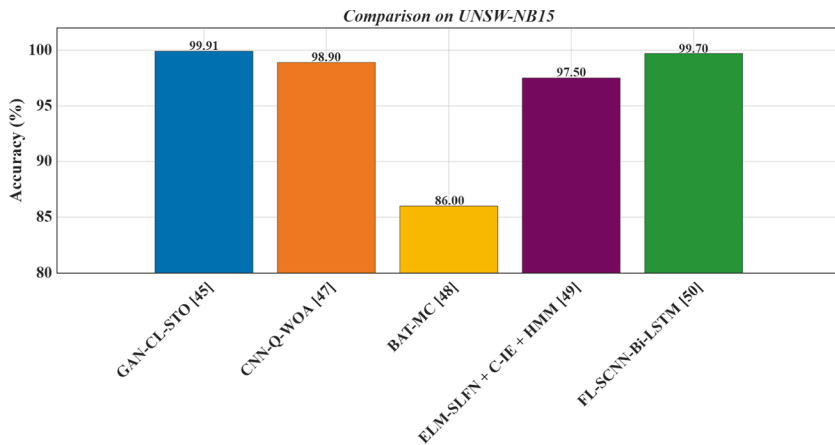


Fig. 12 comparison of the GAN-CL-STO accuracy with other methods using the UNSW-NB15 dataset

method provides good results, its application is limited by its dependency on complete data availability and the fact that it has not been evaluated on the NSL-KDD dataset. Finally, the CNN-Q-WOA model shows reasonable performance but falls short when compared to the improved results achieved by GAN-CL-STO and FL-SCNN-Bi-LSTM.

Figure 10 is a bar graph showing the accuracy of the different methods evaluated on the NSL-KDD dataset. Figure 10 visually compares the performance of these methods in terms of accuracy on the NSL-KDD dataset and shows that GAN-CL-STO [45] and FL-SCNN-Bi-LSTM [50] show the highest accuracy, while BAT-MC [48] has the lowest performance among the presented methods. The accuracy of ELM-SLFN + C-IE + HMM [49] is not shown in this particular comparison.

Figure 11 shows a bar chart comparing the accuracy of several methods on the CIC-IDS2017 dataset. The figure visually shows that GAN-CL-STO [45] and FL-SCNN-Bi-LSTM [50] achieve the highest accuracy, while BAT-MC [48] has the lowest accuracy among the compared methods.

Figure 12 presents a comparative analysis of the accuracy of different methods on the UNSW-NB15 dataset. Figure 12 shows that GAN-CL-STO [45] achieves the highest accuracy with 99.91%, followed by FL-SCNN-Bi-LSTM [50] with 99.70%. BAT-MC [48] has the lowest accuracy of 86.00% among the compared methods.

5 Conclusion

This study introduces the GAN-CL-STO framework, a powerful and efficient intrusion detection system (IDS) developed to secure modern networks. The model achieves a high level of intrusion detection accuracy by combining GAN networks, CL structures, and hyperparameter tuning using the STO algorithm. The GAN-CL-STO model achieved an accuracy of 99.91%, outperforming state-of-the-art methods such as Transformer, GNN, fuzzy systems, reinforcement learning, CNN-LSTM,

PSO-1D CNN, and 1D CNN + BiLSTM. By utilizing optimal feature selection and dynamic tuning of hyperparameters, this framework enables real-time deployment in SDN-based networks while maintaining low computational load. The results are also validated using statistical significance tests, further proving the validity and efficiency of the GAN-CL-STO approach.

6 Limitation

Despite the promising performance of the GAN-CL-STO framework, there are some limitations that need to be overcome in future work:

- *Dataset dependency*: The model has been tested on three benchmark datasets that may not cover all types of network traffic or attack scenarios. Its ability to generalize to other datasets, especially in real-world environments, requires further investigation.
- *Computational complexity*: The optimization step of the STO algorithm can increase the training time, especially with larger populations or more iterations, in more complex or large-scale systems.
- *Scalability*: The current model may face challenges when scaling to support very large network environments with significantly higher traffic volumes and more diverse attack types. Its scalability on large systems and over long periods needs to be investigated.
- *Adversarial Robustness*: Despite the utilization of GANs, the model can still be vulnerable to adversarial attacks, where attackers manipulate input data to trick the system.

7 Future work

Future work on the GAN-CL-STO framework includes testing on different datasets to improve generalization, enhancing hyperparameter optimization using advanced techniques, and deploying in real-time SDN systems. Improving robustness to adversarial attacks and integration with other security mechanisms, like IPS, are also main directions. These efforts aimed to increase the framework's effectiveness for more complex and secure network applications.

In future work, adversarial robustness testing using standard methods such as FGSM and PGD, will be performed to evaluate and enhance the robustness of the framework against carefully crafted perturbations in SDN traffic.

To address this concern, future work will focus on cross-dataset validation and evaluation using real SDN traffic traces to enhance demonstrate the model's robustness and generalization in real-world network conditions.

Acknowledgements Thanks in advance.

Author contribution N.H. and A.D. wrote the main manuscript text and J.R. prepared figures and tables. All authors reviewed the manuscript.

Funding The authors received no financial support for the research, authorship, and/or publication of this article.

Declarations

Conflict of interest The authors have no conflicts of interest to disclose.

Ethical approval This research does not require ethics approval.

Consent to publish This research does not contain any individual person's data.

Data availability The datasets generated and analyzed during the current study are available from the corresponding author on reasonable request.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. Gupta PM (2024) Software-defined networking (SDN): revolutionizing network infrastructure for the future. In: *Software-Defined Network Frameworks*, CRC Press, pp 89–108
2. Hnamte V, Najar AA, Nhung-Nguyen H, Hussain J, Sugali MN (2024) DDoS attack detection and mitigation using deep neural network in SDN environment. *Comput Secur* 138:103661
3. Maleh Y, Qasmaoui Y, El Gholami K, Sadqi Y, Mounir S (2023) A comprehensive survey on SDN security: threats, mitigations, and future directions. *J Reliab Intell Environ* 9(2):201–239
4. Diana L, Dini P, Paolini D (2025) Overview on intrusion detection systems for computers networking security. *Computers* 14(3):87
5. Zhang W, Lazaro JP (2024) A survey on network security traffic analysis and anomaly detection techniques. *Int J Emerg Technol Adv Appl* 1(4):8–16
6. Afzal MU, Abdellatif AA, Zubair M, Mehmood MQ, Massoud Y (2023) Privacy and security in distributed learning: a review of challenges, solutions, and open research issues. *IEEE Access* 11:114562–114581
7. Musa NS, Mirza NM, Rafique SH, Abdallah AM, Murugan T (2024) Machine learning and deep learning techniques for distributed denial of service anomaly detection in software defined networks—current research solutions. *IEEE Access* 12:17982–18011
8. Setitra MA, Fan M, Benkhaddra I, Bensalem ZEA (2024) DoS/DDoS attacks in software defined networks: current situation, challenges and future directions. *Comput Commun*. <https://doi.org/10.1016/j.comcom.2024.04.035>
9. Ahmad S, Mir AH (2021) Scalability, consistency, reliability and security in SDN controllers: a survey of diverse SDN controllers. *J Netw Syst Manage* 29:1–59
10. Etchezarreta X, Garitano I, Iturbe M, Zurutuza U (2023) Software-defined networking approaches for intrusion response in Industrial Control Systems: a survey. *Int J Crit Infrastruct Prot* 42:100615
11. Mishra SR, Shanmugam B, Yeo KC, Thennadil S (2025) SDN-enabled IoT security frameworks—a review of existing challenges. *Technologies* 13(3):121

12. Hirsi A et al (2025) Comprehensive analysis of DDoS anomaly detection in software-defined networks. IEEE Access. <https://doi.org/10.1109/ACCESS.2025.3535943>
13. Yaghoubi E, Yaghoubi E, Khamees A, Razmi D, Lu T (2024) A systematic review and meta-analysis of machine learning, deep learning, and ensemble learning approaches in predicting EV charging behavior. *Eng Appl Artif Intell* 135:108789
14. Thakkar A, Lohiya R (2021) A review on machine learning and deep learning perspectives of IDS for IoT: recent updates, security issues, and challenges. *Arch Comput Methods Eng* 28(4):3211–3243
15. Hassan HA, Hemdan EE, El-Shafai W, Shokair M, Abd El-Samie FE (2024) Detection of attacks on software defined networks using machine learning techniques and imbalanced data handling methods. *Secur Privacy* 7(2):e350
16. Arevalo-Herrera J, Camargo Mendoza JE, Martinez Torre JI (2022) Network anomaly detection with machine learning techniques for sdn networks. In: *Proceedings of the 7th International Conference on Information and Education Innovations*, pp 129–135
17. Elsayed MS, Le-Khac N-A, Dev S, Jurcut AD (2019) Machine-learning techniques for detecting attacks in SDN. In: *2019 IEEE 7th International Conference on Computer Science and Network Technology (ICCSNT)*, pp 277–281
18. D'Angelo G, Palmieri F (2021) Network traffic classification using deep convolutional recurrent autoencoder neural networks for spatial–temporal features extraction. *J Netw Comput Appl* 173:102890
19. Yaghoubi E, Yaghoubi E, Yusupov Z, Maghami MR (2024) A real-time and online dynamic reconfiguration against cyber-attacks to enhance security and cost-efficiency in smart power microgrids using deep learning. *Technologies* 12(10):197
20. Yaghoubi E, Yaghoubi E, Khamees A, Vakili AH (2024) A systematic review and meta-analysis of artificial neural network, machine learning, deep learning, and ensemble learning approaches in field of geotechnical engineering. *Neural Comput Appl* 1–45
21. Mehdi SS (2024) Cybersecurity Of Cyber-Physical Systems Using Machine Learning Approach. College of Electrical & Mechanical Engineering (CEME), NUST
22. Bahar AAM, Ferrahi KS, Messai M-L, Seba H, Amrouche K (2025) “CONTINUUM: Detecting APT attacks through spatial-temporal graph neural networks,” *arXiv Prepr. arXiv2501.02981*
23. Ding H, Chen L, Dong L, Fu Z, Cui X (2022) Imbalanced data classification: a KNN and generative adversarial networks-based hybrid approach for intrusion detection. *Futur Gener Comput Syst* 131:240–254
24. Kostage K, West D, Meinert T, Qu C, Calyam P, Mazzola L (2024) Enhancing Autonomous Intrusion Detection System with Generative Adversarial Networks,” in *2024 IEEE 20th International Conference on e-Science (e-Science)*, pp 1–10
25. Vaz B, Figueira Á (2024) Gans in the panorama of synthetic data generation methods. *ACM Trans Multimed Comput Commun Appl* 21(1):1–28
26. Agrawal G, Kaur A, Myneni S (2024) A review of generative models in generating synthetic attack data for cybersecurity. *Electronics* 13(2):322
27. Rayavarapu SM, Tammineni SP, Gottapu SR, Singam A (2024) A review of generative adversarial networks for security applications. *Informatyka, Automatyka, Pomiar w Gospodarce i Ochronie Środowiska* 14(2):66–70
28. Ahmadi SS, Rashad S, Elgazzar H (2019) Efficient feature selection for intrusion detection systems. In: *2019 IEEE 10th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*, pp 1029–1034
29. Aljehane NO, Mengash HA, Hassine SBH, Alotaibi FA, Salama AS, Abdelbagi S (2024) Optimizing intrusion detection using intelligent feature selection with machine learning model. *Alex Eng J* 91:39–49
30. Mostafa RR, El-Attar NE, Sabbah SF, Vidyarthi A, Hashim FA (2023) ST-AL: a hybridized search based metaheuristic computational algorithm towards optimization of high dimensional industrial datasets. *Soft Comput* 27(18):13553–13581
31. Pravin PS, Tan JZM, Yap KS, Wu Z (2022) Hyperparameter optimization strategies for machine learning-based stochastic energy efficient scheduling in cyber-physical production systems. *Digital Chem Eng* 4:100047
32. Gunes F, Czika WA, Haller SE, Sglavo U (2020) System for automatic, simultaneous feature selection and hyperparameter tuning for a machine learning model. *Google Patents*, Mar. 24, 2020

33. De Carvalho HDP, Soares WL, Santos WB, Fagundes R (2022) A comparison study about parameter optimization using swarm algorithms. *IEEE Access* 10:55488–55498
34. Divasón J, Pernia-Espinoza A, Martínez-de-Pison FJ (2022) New hybrid methodology based on particle swarm optimization with genetic algorithms to improve the search of parsimonious models in high-dimensional databases. In: *International Conference on Hybrid Artificial Intelligence Systems*, pp 335–347.
35. Zhang D, Wang J, Gao H, Ni Z, Zhang H (2024) Network security anomaly node detection based on graph neural network and attention mechanism
36. Zhang H, Cao T (2024) a hybrid approach to network intrusion detection based on graph neural networks and transformer architectures. In: *2024 14th International Conference on Information Science and Technology (ICIST)*, pp 574–582
37. Lakha B, Mount SL, Serra E, Cuzzocrea A (2022) Anomaly detection in cybersecurity events through graph neural network and transformer based model: A case study with beth dataset. In: *2022 IEEE International Conference on Big Data (Big Data)*, pp 5756–5764
38. Latif H, Suárez-Varela J, Cabellos-Aparicio A, Barlet-Ros P (2023) Detecting contextual network anomalies with graph neural networks. In: *Proceedings of the 2nd on Graph Neural Networking Workshop 2023*, pp 25–30
39. Ahmed N (2023) On The Practicality of Blockchain-based Security and Privacy for Next Generation SDN. In: *2023 10th International Conference on Wireless Networks and Mobile Communications (WINCOM)*, 2023, pp 1–6
40. Alrashde H, Eassa F, Marish Ali A, Albalwy F, Aljihani H (2024) A blockchain-based security framework for East-West interface of SDN. *Electronics* 13(19):3799
41. Pahlevan M, Ionita V (2022) Secure and efficient exchange of threat information using blockchain technology. *Information* 13(10):463
42. Seungwon S, Seungwon WOO (2020) System for Secure Software Defined Networking Based on Block-Chain and Method Thereof. Google Patents, May 28, 2020
43. Almakhour M, Wehby A, Sliman L, Samhat AE, Mellouk A (2021) “Smart contract based solution for secure distributed sdn. In: *2021 11th ifip international conference on new technologies, mobility and security (ntms)*, pp 1–6
44. Rajathi N, Saritha G, VJ Ramya (2024) Adaptive intrusion detection in cyber-physical systems using reinforcement learning-based autoencoders. In: *2024 International Conference on Integrated Intelligence and Communication Systems (ICIICS)*, pp 1–7
45. Lee W, Cabrera JBD, Thomas A, Balwalli N, Saluja S, Zhang Y (2002) Performance adaptation in real-time intrusion detection systems. In: *Recent Advances in Intrusion Detection: 5th International Symposium, RAID 2002 Zurich, Switzerland, October 16–18, 2002 Proceedings 5*, pp 252–273
46. Sethi K, Kumar R, Prajapati N, Bera P (2020) Deep reinforcement learning based intrusion detection system for cloud infrastructure. In: *2020 International Conference on COMMunication Systems & NETWORKS (COMSNETS)*, pp 1–6
47. Baazeem R. “Multilayered framework for enhancing data confidentiality, integrity, and threat detection through blockchain, advanced cryptography, and machine learning”
48. Bose S, Gokulraj G, Maheswaran N, Logeswari G, Anitha T, Prabhu D (2024) Multi-layered security framework for intrusion detection system in software defined networking environment using machine learning. In: *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pp 1–7
49. Bour H, Abolhasan M, Jafarizadeh S, Lipman J, Makhdoom I (2022) A multi-layered intrusion detection system for software defined networking. *Comput Electr Eng* 101:108042
50. Bukhari SMS et al (2024) Secure and privacy-preserving intrusion detection in wireless sensor networks: Federated learning with SCNN-Bi-LSTM for enhanced reliability. *Ad Hoc Netw* 155:103407

Authors and Affiliations

Naseer Hameed Saadoon Al-Sarray¹ · Ayşe Demirhan¹ · Javad Rahebi²

✉ Javad Rahebi
cevatrahebi@topkapi.edu.tr

Naseer Hameed Saadoon Al-Sarray
nasirhamed8@gmail.com

Ayşe Demirhan
ayseoguz@gazi.edu.tr

¹ Department of Electrical and Electronics Engineering, Gazi University, 06560 Ankara, Türkiye

² Department of Software Engineering, Istanbul Topkapi University, 34662 Istanbul, Türkiye